

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA1407

COURSE NAME: Compiler Design for High-Level Programming

COURSE OUTCOME COVERED

CO3: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)*

Assessment Tool Weightage: 10%

QUESTIONS: 60

Total Marks:60

Annexure A

MCQ

Code of Conduct:

I M.V.Rakesh Reddy/192524164 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M.V.Rakesh Reddy

Criteria	Weightage	Level 4 – Exemplary (50–60 Marks)	Level 3 – Proficient (40–49 Marks)	Level 2 – Developing (30–39 Marks)	Level 1 – Beginning (0–29 Marks)
Number of Correct Answers	100%	50 to 60 correct answers	40 to 49 correct answers	30 to 39 correct answers	0 to 29 correct answers
Accuracy	100%	Excellent (90–100%)	Good (75–89%)	Average (50–74%)	Poor (Below 50%)

Course Faculty

Annexure A

MCQ

1. A compiler designer wants to attach semantic meaning to grammar productions in order to generate intermediate representations efficiently. Which compiler concept is most suitable for this purpose?

- A) Lexical buffering
- B) Associating semantic rules with grammar productions**
- C) Register allocation
- D) Peephole optimization

ANSWER:

B) Associating semantic rules with grammar productions

2. While compiling an arithmetic expression, a compiler internally creates a hierarchical representation showing operators and operands. Which structure is being generated?

- A) Stack frame
- B) Queue structure
- C) Syntax Tree**
- D) Linked List

ANSWER

C) Syntax Tree

3. A compiler evaluates attributes from child nodes before computing the parent node value during semantic analysis. Which type of attribute evaluation is taking place?

- A) Parent-driven evaluation
- B) Synthesized attribute evaluation**
- C) Inherited attribute evaluation
- D) Dynamic attribute evaluation

ANSWER

B) Synthesized attribute evaluation

4. A language processor uses LR parsing to evaluate semantic information during reductions. Which attribute grammar is naturally suitable in this case?

- A) Context-free attribute grammar
- B) Recursive attribute grammar
- C) S-attributed definition**
- D) Circular attribute grammar

ANSWER

5. During semantic rule evaluation, a parser requires both parent information and child information while traversing grammar symbols from left to right. Which definition supports this requirement?

- A) S-attributed definition
 - B) Static semantic definition
 - C) L-attributed definition**
 - D) Ambiguous definition
-

6. A compiler engineer wants child nodes to be processed before the parent node during semantic evaluation of expressions. Which traversal strategy should be selected?

- A) Preorder traversal
 - B) Inorder traversal
 - C) Postorder traversal**
 - D) Level-order traversal
-

7. An expression such as $(a+b)*c$ is represented internally without unnecessary grammar details. What does this representation mainly capture?

- A) Memory layout
 - B) Hierarchical program structure**
 - C) Register allocation
 - D) CPU execution order
-

8. While evaluating semantic rules, information is passed from a parent node toward its child nodes. Which type of attribute is involved?

- A) Synthesized attribute
 - B) Terminal attribute
 - C) Inherited attribute**
 - D) Dynamic attribute
-

9. A compiler rejects the statement `int x = "hello";` before execution begins. Which feature of the programming language is responsible for detecting this issue?

- A) Token generation
 - B) Type system**
 - C) Syntax optimization
 - D) Loop unrolling
-

10. Consider the statement `float value = 25;`. Which compiler activity occurs automatically here?

- A) Explicit casting
 - B) Dynamic conversion
 - C) Implicit type conversion**
 - D) Symbol replacement
-

11. A semantic analyzer verifies whether variables are declared and operations are type-compatible. Which compiler phase performs these tasks?

- A) Lexical Analysis
 - B) Syntax Analysis
 - C) Semantic Analysis**
 - D) Linking
-

12. A compiler developer chooses a representation that eliminates unnecessary grammar symbols while preserving expression meaning. Which structure is preferred over a parse tree?

- A) Symbol Table
 - B) Syntax Tree**
 - C) Token Stream
 - D) Finite Automaton
-

13. During compilation, all information regarding identifiers, scope, and datatype is maintained centrally. Which compiler component performs this responsibility?

- A) Code Generator
 - B) Symbol Table**
 - C) Syntax Parser
 - D) Optimizer
-

14. A compiler uses only synthesized information for semantic evaluation of grammar productions. Which type of attribute grammar is being implemented?

- A) Dynamic grammar
 - B) L-attributed grammar
 - C) S-attributed grammar**
 - D) Context-sensitive grammar
-

15. A recursive descent parser performs semantic translation while expanding productions from the start symbol downward. Which translation approach is being followed?

- A) Bottom-up translation
 - B) Top-down translation**
 - C) Shift-reduce translation
 - D) Operator precedence translation
-

16. During shift-reduce parsing, intermediate grammar symbols are temporarily stored before reductions occur. Which data structure supports this operation most effectively?

- A) Queue
 - B) Heap
 - C) Stack**
 - D) Linked List
-

17. A compiler reports an error when a boolean value is added to an integer variable. What compiler mechanism detected this problem?

- A) Syntax checking
 - B) Type checking**
 - C) Token validation
 - D) Memory management
-

18. A programmer declares a variable to store whole numbers without decimal values. Which datatype is most appropriate?

- A) Boolean
 - B) Integer**
 - C) Character
 - D) String
-

19. Two array types are considered equivalent because they possess identical internal structures even though their names differ. Which type equivalence principle is applied?

- A) Name equivalence
 - B) Structural equivalence**
 - C) Functional equivalence
 - D) Dynamic equivalence
-

20. A compiler implementing LR parsing evaluates semantic rules during reductions. Which evaluation strategy is best suited for this implementation?

- A) Top-down evaluation
 - B) Parallel evaluation
 - C) Bottom-up evaluation**
 - D) Recursive evaluation
-

21. During expression analysis, a compiler stores operators as internal nodes and operands as leaves. Which representation is being used?

- A) AVL Tree
 - B) Heap Tree
 - C) Syntax Tree**
 - D) B-Tree
-

22. A semantic rule requires information from symbols appearing to the left side of a production. Which attribute grammar supports such evaluation?

- A) S-attributed grammar
 - B) L-attributed grammar**
 - C) Ambiguous grammar
 - D) Recursive grammar
-

23. After semantic analysis, a compiler converts validated source constructs into machine-independent instructions. Which compiler phase performs this task?

- A) Lexical Analysis
 - B) Intermediate Code Generation**
 - C) Linking
 - D) Parsing
-

24. A programmer intentionally converts a floating-point value into an integer using a language construct. What is this process called?

- A) Promotion
 - B) Reduction
 - C) Casting**
 - D) Coercion
-

25. While scanning source code, a compiler groups characters into meaningful units such as keywords and identifiers. Which phase performs this operation?

- A) Semantic Analysis
 - B) Syntax Analysis
 - C) Lexical Analysis**
 - D) Optimization
-

26. A parse tree generated for a program statement explicitly contains all grammar symbols used in derivation. What is the primary characteristic of this structure?

- A) Contains machine instructions
 - B) Represents grammar symbols**
 - C) Removes terminals
 - D) Stores registers only
-

27. A bottom-up parser starts constructing derivations beginning with identifiers and constants before reaching the start symbol. Which nodes are processed first?

- A) Root nodes
 - B) Internal nodes
 - C) Leaf nodes**
 - D) Header nodes
-

28. During assignment checking, a compiler ensures the datatype of the expression matches the datatype of the variable receiving it. Which component performs this verification?

- A) Loader
 - B) Semantic checker**
 - C) Tokenizer
 - D) Syntax formatter
-

29. Two variables are treated as equivalent only because they were declared using the same type name. Which equivalence mechanism is being followed?

- A) Structural equivalence
 - B) Dynamic equivalence
 - C) Name equivalence**
 - D) Functional equivalence
-

30. A semantic evaluator computes all child expressions before processing the parent expression. Which traversal technique reflects this behavior?

- A) Preorder traversal
 - B) Inorder traversal
 - C) Postorder traversal**
 - D) Reverse traversal
-

31. A compiler designer inserts executable semantic instructions directly inside grammar productions to perform translation tasks. What is this mechanism called?

A) Syntax-directed translation scheme

B) Token generation mechanism

C) Recursive grammar scheme

D) DFA representation

32. A parser repeatedly performs shift and reduce operations until the start symbol is produced. Which parsing strategy is being used?

A) Predictive parsing

B) Top-down parsing

C) Bottom-up parsing

D) Recursive parsing

33. A strongly typed programming language prevents invalid arithmetic operations between incompatible datatypes. Which language feature enforces this?

A) Type system

B) Code optimizer

C) Symbol allocator

D) Loader

34. A programmer creates a collection capable of storing multiple values of the same datatype under one name. Which datatype category does this belong to?

A) Primitive datatype

B) Derived datatype

C) Boolean datatype

D) Character datatype

35. In a syntax tree representing an assignment statement, the highest node corresponds to the complete statement meaning. Which element does the root node generally represent?

- A) Constant value
 - B) Identifier name
 - C) Entire expression or statement**
 - D) Memory address
-

36. A compiler reports an error because a variable is used outside its declared scope. Which compiler phase detects this issue?

- A) Syntax Analysis
 - B) Semantic Analysis**
 - C) Code Optimization
 - D) Linking
-

37. A compiler automatically converts an integer operand into a floating-point operand before arithmetic evaluation. What type of conversion is this?

- A) Narrowing conversion
 - B) Implicit widening conversion**
 - C) Explicit casting
 - D) Recursive conversion
-

38. During attribute evaluation, semantic rules are processed from left to right so that inherited information becomes available when required. Which attribute grammar follows this restriction?

- A) Circular grammar
 - B) L-attributed grammar**
 - C) S-attributed grammar
 - D) Ambiguous grammar
-

39. A compiler uses shift-reduce actions and reduction tables to parse complex programming constructs. Which parser is most likely being used?

- A) Predictive parser
 - B) Recursive descent parser
 - C) LR parser**
 - D) LL(1) parser
-

40. A compiler engineer associates semantic computations directly with grammar rules for translation purposes. To what are these semantic rules attached?

- A) Tokens only
 - B) Grammar productions**
 - C) Machine registers
 - D) Memory blocks
-

41. During semantic evaluation, a node computes its value entirely using values obtained from child nodes. Which attribute type is involved?

- A) Inherited attribute
 - B) Dynamic attribute
 - C) Synthesized attribute**
 - D) External attribute
-

42. A compiler designer prefers a compact internal representation that removes unnecessary non-terminal symbols from derivations. Which representation is best suited?

- A) Parse Tree
 - B) Syntax Tree**
 - C) NFA
 - D) Transition Graph
-

43. A program contains the statement `total=amount + tax;` where `tax` has never been declared. Which compiler phase identifies this error?

- A) Lexical Analysis
 - B) Parsing
 - C) Semantic Analysis**
 - D) Code Generation
-

44. A compiler automatically changes the datatype of an operand during expression evaluation to avoid type mismatch. What is this process called?

- A) Coercion**
 - B) Parsing
 - C) Reduction
 - D) Folding
-

45. Two structures are treated as equivalent because they contain identical field arrangements and datatypes. Which equivalence principle is being applied?

- A) Name equivalence
 - B) Scope equivalence
 - C) Structural equivalence**
 - D) Dynamic equivalence
-

46. A compiler implementing LR parsing evaluates semantic actions during reduce operations. Which evaluation strategy aligns best with this parser behavior?

- A) Predictive evaluation
 - B) Recursive evaluation
 - C) Bottom-up evaluation**
 - D) Top-down evaluation
-

47. A compiler developer chooses an attribute grammar that can be evaluated naturally during reduce actions without inherited information. Which grammar type is most appropriate?

- A) L-attributed grammar
 - B) S-attributed grammar**
 - C) Ambiguous grammar
 - D) Circular grammar
-

48. In a syntax tree for arithmetic expressions, the terminal values such as variables and constants appear at which location?

- A) Internal nodes
 - B) Root node
 - C) Leaf nodes**
 - D) Header nodes
-

49. A programmer explicitly converts a floating-point result into an integer before storing it in a variable. Which operation is being applied?

- A) Promotion
 - B) Casting**
 - C) Translation
 - D) Coercion
-

50. After successful syntax checking, a compiler begins verifying meanings, datatypes, and scope rules. Which phase starts next?

- A) Lexical Analysis
 - B) Semantic Analysis**
 - C) Linking
 - D) Loading
-

51. A programming language verifies datatype compatibility during compilation and sometimes again during execution. What does this indicate about type checking?

- A) It occurs only during loading
 - B) It occurs only during runtime
 - C) It can occur during compile time and runtime**
 - D) It is unnecessary in modern languages
-

52. A parser constructs derivations beginning from tokens and gradually reduces them until the start symbol is obtained. Which parsing approach is demonstrated?

- A) Top-down parsing
 - B) Predictive parsing
 - C) Bottom-up parsing**
 - D) Recursive parsing
-

53. A compiler internally represents expressions using parent-child relationships between operators and operands instead of linear text order. Which representation style is being followed?

- A) Sequential representation
 - B) Hierarchical representation**
 - C) Tabular representation
 - D) Circular representation
-

54. A compiler phase determines whether expressions, assignments, and declarations carry valid meaning according to language rules. Which aspect is primarily analyzed?

- A) Syntax structure
 - B) Meaning of constructs**
 - C) Register allocation
 - D) Instruction scheduling
-

55. Two datatype declarations are accepted as compatible because the language rules consider them logically interchangeable. What does this describe?

- A) Operator precedence
- B) Variable scope
- C) Type equivalence**
- D) Memory allocation

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (90–100 Marks)	Level 3 – Proficient (75–89 Marks)	Level 2 – Developing (50–74 Marks)	Level 1 – Beginning (0–49 Marks)
Number of Correct Answers	100%	90 to 100 correct answers	75 to 89 correct answers	50 to 74 correct answers	0 to 49 correct answers

Criteria	Weightage (%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer

Criteria	Weight age (%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Application of Knowledge	20	Correctly applies concepts to solve the question/prob lem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understand able with minor issues	Somewhat unclear or poorly structured	Difficult to understand

ANSWERS

1. B) Associating semantic rules with grammar productions

Explanation: Semantic rules attached to grammar productions form the basis of syntax-directed definitions/translation.

2. C) Syntax Tree

Explanation: A syntax tree represents operators and operands in a hierarchical program structure.

3. B) Synthesized attribute evaluation

Explanation: A synthesized attribute is computed from information obtained from child nodes.

4. C) S-attributed definition

Explanation: S-attributed definitions use only synthesized attributes and fit LR reductions naturally.

5. C) L-attributed definition

Explanation: L-attributed definitions support inherited information with left-to-right dependency restrictions.

6. C) Postorder traversal

Explanation: Postorder visits children before the parent, matching bottom-up semantic evaluation.

7. B) Hierarchical program structure

Explanation: A syntax tree captures the meaningful hierarchical relationships among program constructs.

8. C) Inherited attribute

Explanation: Inherited attributes carry contextual information from a parent or left sibling toward a child.

9. B) Type system

Explanation: A static type system can reject incompatible assignments such as an integer receiving a string.

10. C) Implicit type conversion

Explanation: The integer 25 can be automatically promoted to float for a floating-point variable.

11. C) Semantic Analysis

Explanation: Semantic analysis checks declarations, types, scope and meaning after syntax is recognized.

12. B) Syntax Tree

Explanation: A syntax tree removes unnecessary grammar symbols while preserving the essential meaning.

13. B) Symbol Table

Explanation: The symbol table stores identifiers together with scope, type and related semantic information.

14. C) S-attributed grammar

Explanation: An S-attributed grammar contains only synthesized attributes.

15. B) Top-down translation

Explanation: Recursive-descent translation expands the grammar from the start symbol toward the input leaves.

16. C) Stack

Explanation: Shift-reduce parsers use a stack to store grammar symbols and parser states during reductions.

17. B) Type checking

Explanation: Type checking detects invalid operations such as adding a boolean to an integer.

18. B) Integer

Explanation: Integer is the standard datatype for whole-number values without fractional parts.

19. B) Structural equivalence

Explanation: Structural equivalence considers types equivalent when their internal structures match.

20. C) Bottom-up evaluation

Explanation: LR parsing proceeds through reductions, so semantic evaluation naturally proceeds bottom-up.

21. C) Syntax Tree

Explanation: In a syntax tree, operators are internal nodes and operands commonly appear as leaves.

22. B) L-attributed grammar

Explanation: L-attributed definitions allow dependencies on inherited information and attributes of symbols to the left.

23. B) Intermediate Code Generation

Explanation: This phase converts validated source structures into machine-independent intermediate instructions.

24. C) Casting

Explanation: An explicit programmer-requested conversion, such as float to integer, is called casting.

25. C) Lexical Analysis

Explanation: Lexical analysis groups source characters into tokens such as keywords and identifiers.

26. B) Represents grammar symbols

Explanation: A parse tree explicitly records the grammar symbols used in the derivation.

27. C) Leaf nodes

Explanation: Bottom-up construction begins with terminals such as identifiers and constants at the leaves.

28. B) Semantic checker

Explanation: Assignment compatibility is a semantic property verified by the semantic checker.

29. C) Name equivalence

Explanation: Name equivalence treats types as equivalent based on the same declared type identity/name.

30. C) Postorder traversal

Explanation: Postorder processing evaluates all children before the parent expression.

31. A) Syntax-directed translation scheme

Explanation: A translation scheme places executable semantic actions directly within grammar productions.

32. C) Bottom-up parsing

Explanation: Shift-reduce parsing starts from input symbols and reduces them toward the start symbol.

33. A) Type system

Explanation: A strong type system enforces rules preventing invalid operations between incompatible types.

34. B) Derived datatype

Explanation: A collection such as an array is a derived/constructed datatype rather than a primitive value.

35. C) Entire expression or statement

Explanation: The root of a syntax tree represents the highest-level construct being represented.

36. B) Semantic Analysis

Explanation: Scope violations concern program meaning and declarations, so semantic analysis detects them.

37. B) Implicit widening conversion

Explanation: Converting an integer to floating point automatically is a widening promotion.

38. B) L-attributed grammar

Explanation: L-attributed definitions permit inherited information to be evaluated in a left-to-right order.

39. C) LR parser

Explanation: LR parsers use shift-reduce actions and reduction tables to recognize a broad class of grammars.

40. B) Grammar productions

Explanation: Syntax-directed semantic rules are associated with grammar productions.

41. C) Synthesized attribute

Explanation: A synthesized attribute is calculated entirely from attributes of child nodes.

42. B) Syntax Tree

Explanation: A syntax tree is a compact representation that removes unnecessary nonterminal grammar details.

43. C) Semantic Analysis

Explanation: Use of an undeclared identifier is a semantic error detected through symbol-table checking.

44. A) Coercion

Explanation: Automatic conversion of an operand to another type during evaluation is called coercion.

45. C) Structural equivalence

Explanation: Structures with matching fields and datatypes can be considered equivalent structurally.

46. C) Bottom-up evaluation

Explanation: Semantic actions during LR reduce operations naturally follow bottom-up evaluation.

47. B) S-attributed grammar

Explanation: S-attributed definitions require only synthesized information, making them ideal for reductions.

48. C) Leaf nodes

Explanation: Variables and constants are terminal values and therefore normally occur at syntax-tree leaves.

49. B) Casting

Explanation: An explicit float-to-integer conversion performed by the programmer is a cast.

50. B) Semantic Analysis

Explanation: After syntax checking, the compiler verifies types, scope and the meaning of constructs.

51. C) It can occur during compile time and runtime

Explanation: Type checking may be static, dynamic, or a combination depending on the language.

52. C) Bottom-up parsing

Explanation: Reducing tokens toward the start symbol is the defining direction of bottom-up parsing.

Quick Revision: S-attributed → synthesized only → bottom-up/LR; L-attributed → inherited + synthesized with left-to-right dependencies → top-down friendly; syntax tree → compact meaningful structure; semantic analysis →

Explanation: Parent-child relationships among operators and operands form a hierarchical representation. types, scope and declarations; structural equivalence → same structure; name equivalence → same declared type

identity; coercion → implicit conversion; casting → explicit conversion.

54. B) Meaning of constructs

Explanation: Semantic analysis determines whether declarations, expressions and assignments are meaningful.

55. C) Type equivalence

Explanation: Type equivalence describes the language rules that determine whether two types are compatible/interchangeable.